1. Presentación de la Asignatura

Sistemas Distribuidos es una asignatura de formación profesional específica del séptimo semestre del programa de Ingeniería de Sistemas. Con una carga de 144 horas —64 horas presenciales y 80 horas de trabajo autónomo—, la asignatura responde a la demanda actual del mercado tecnológico por profesionales capaces de diseñar, implementar y operar sistemas de software distribuidos, escalables y resilientes.

El curso se estructura en torno a tres grandes ejes temáticos que se desarrollan de manera progresiva, llevando al estudiante desde los conceptos fundamentales hasta la implementación de un sistema distribuido completo y operativo en entornos reales de producción:

- Fundamentos y Arquitecturas en Sistemas Distribuidos
- Contenerización, Redes y Persistencia
- Operaciones, Seguridad y Despliegue en la Nube

2. Objetivo General y Resultados de Aprendizaje

2.1 Objetivo General

Brindar al estudiante las bases del lenguaje de programación para aplicar estructuras de sistemas distribuidos, mejorando la escalabilidad, accesibilidad, liberación de carga y estabilidad del proyecto de software conforme a los requerimientos del cliente.

2.2 Resultado de Aprendizaje Esperado (RAA)

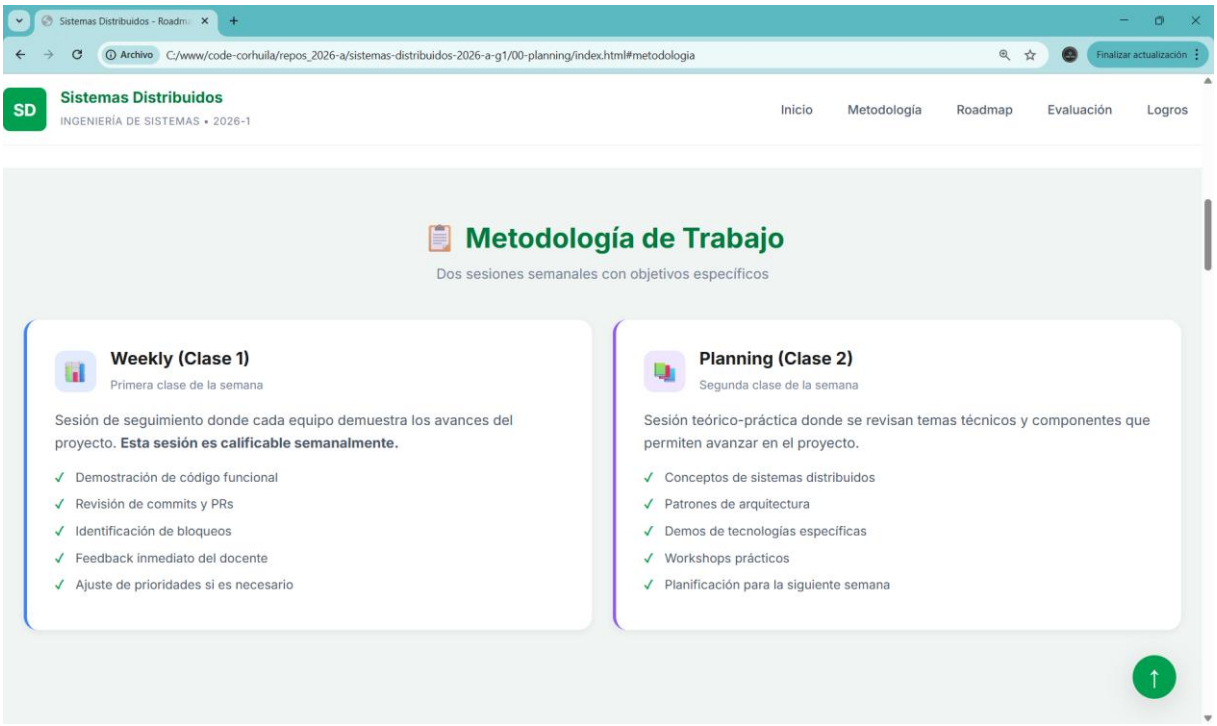
> *"Aplica técnicas, habilidades y herramientas modernas de los Sistemas Distribuidos con criterios de calidad e integración."*

2.3 Objetivos Específicos

1. Dominar los conceptos clave de los sistemas distribuidos a nivel de hardware y software.
2. Desarrollar soluciones eficientes mediante la comprensión profunda del problema y el uso de arquitecturas distribuidas.
3. Aplicar modelos de arquitectura distribuida para resolver problemas reales y empresariales usando lenguajes de programación de alto nivel.

3. Metodología

La metodología combina estrategias activas de aprendizaje con el desarrollo de un proyecto real de ingeniería, organizado en sprints de trabajo semanales. Se aplica un enfoque de Aprendizaje Basado en Proyectos (ABP) con gamificación a través de insignias de progreso académico.



3.1 Enfoque Pedagógico

- Clases magistrales participativas con demostraciones técnicas en vivo.
- Talleres prácticos de implementación guiada.
- Desarrollo iterativo de un proyecto distribuido real (MVP incremental).
- Revisión de código y retroalimentación continua entre pares y docente.
- Documentación de decisiones arquitectónicas mediante ADRs (Architecture Decision Records).

3.2 Sistema de Insignias — Gamificación

El avance del estudiante se visualiza mediante tres insignias progresivas que agrupan los contenidos del semestre:

- **Insignia 1 — Arquitecturas Distribuidas:** Modelos distribuidos, IPC, RPC, mensajería y relojes lógicos de sincronización.
- **Insignia 2 — Comunicación y Consistencia:** Transacciones distribuidas, modelos de consistencia, replicación, Teorema CAP, algoritmos de consenso Paxos y Raft.
- **Insignia 3 — Tolerancia a Fallos y Nube:** Estrategias de replicación, computación en la nube y arquitecturas resilientes.

3.3 Flujo de Trabajo de Desarrollo

Cada equipo trabaja con una estrategia de ramas en GitHub que simula entornos reales de producción:

- Rama develop → Entorno DEV (desarrollo activo)

- Rama qa → Entorno QA (pruebas de integración y validación)
- Rama main → Entorno PRODUCCIÓN (versión estable y validada)

Todos los cambios se integran mediante Merge Requests (MR), asegurando revisión de código y control de calidad continuo. Las hotfixes para cambios críticos tienen un flujo de integración rápida documentado.

3.4 Sistema de Evaluación

La evaluación se distribuye en tres cortes académicos con componentes de autoevaluación, coevaluación y evaluación docente:

Corte	Autoevaluación	Coevaluación	Evaluación Corhuila / ScotiaTech
Primer Corte (30%)	5%	5%	20%
Segundo Corte (30%)	5%	5%	20%
Tercer Corte (40%)	5%	5%	30%

Los instrumentos de evaluación incluyen: cuestionarios tipo Saber Pro, informes escritos individuales y grupales, evaluaciones prácticas de implementación, revisiones de código y evidencias en GitHub.

4. Plan de Trabajo — Unidades Temáticas

El semestre se organiza en 16 semanas distribuidas en tres unidades de aprendizaje. Cada semana contempla sesiones presenciales de teoría y práctica, complementadas con actividades de trabajo autónomo.

Corte 1 — Primer Corte

Semanas 1 - 5 | 30% del curso

100%
5/5 semanas

Fundamentos y Setup
Introducción a Sistemas Distribuidos

- WEEKLY — **Formación de equipos**: Presentación, equipos de trabajo, lluvia de ideas de proyectos
- PLANNING — **Fundamentos SD**: Teorema CAP, arquitecturas distribuidas, patrones básicos

Git · GitHub

Arquitectura del Proyecto
Diseño de Microservicios

- WEEKLY — **Definición del proyecto**: User stories, arquitectura inicial, backlog del producto
- PLANNING — **Microservicios**: Patrones de diseño, Domain-Driven Design, bounded contexts

DDD · Arquitectura

Repositorio y Ambientes
Control de Versiones Avanzado

- WEEKLY — **Setup del repositorio**: Estructura monorepo/multirepo, branches develop/qa/main
- PLANNING — **API Gateway**: Service Discovery, comunicación síncrona, REST APIs

Git Flow · GitHub Actions

Primer Microservicio
Java Spring Boot

- WEEKLY — **Microservicio Java**: Desarrollo del primer microservicio con Spring Boot
- PLANNING — **PostgreSQL**: Bases de datos relacionales en microservicios, migraciones

Java · Spring Boot · PostgreSQL

⭐ ENTREGA 1: MVP
Primera Entrega de Producto

- WEEKLY — **Presentación MVP**: Demo del MVP: API funcional básica con al menos un microservicio
- PLANNING — **Retroalimentación**: Feedback, métricas de calidad, planificación segundo corte

Demo · MVP · Evaluación

Corte 2 — Segundo Corte

Semanas 6 - 10 | 35% del curso

60%
3/5 semanas

6 Base de Datos NoSQL
MongoDB en Microservicios

- WEEKLY — **Avances post-MVP**: Mejoras basadas en feedback, refactoring inicial
- PLANNING — **MongoDB**: Modelado NoSQL, agregaciones, índices para microservicios

MongoDB · NoSQL

Segundo Microservicio
Integración MongoDB

- WEEKLY — **Micro con MongoDB**: Desarrollo de microservicio con base de datos NoSQL
- PLANNING — **Mensajería Asíncrona**: Patrones de comunicación, eventos, pub/sub

MongoDB · Eventos

Comunicación Asíncrona
Message Brokers

- WEEKLY — **Implementación mensajería**: RabbitMQ o Kafka para comunicación entre servicios
- PLANNING — **Docker Básico**: Contenedorización, Dockerfiles, buenas prácticas

RabbitMQ · Kafka · Docker

Contenedorización
Docker y Orquestación

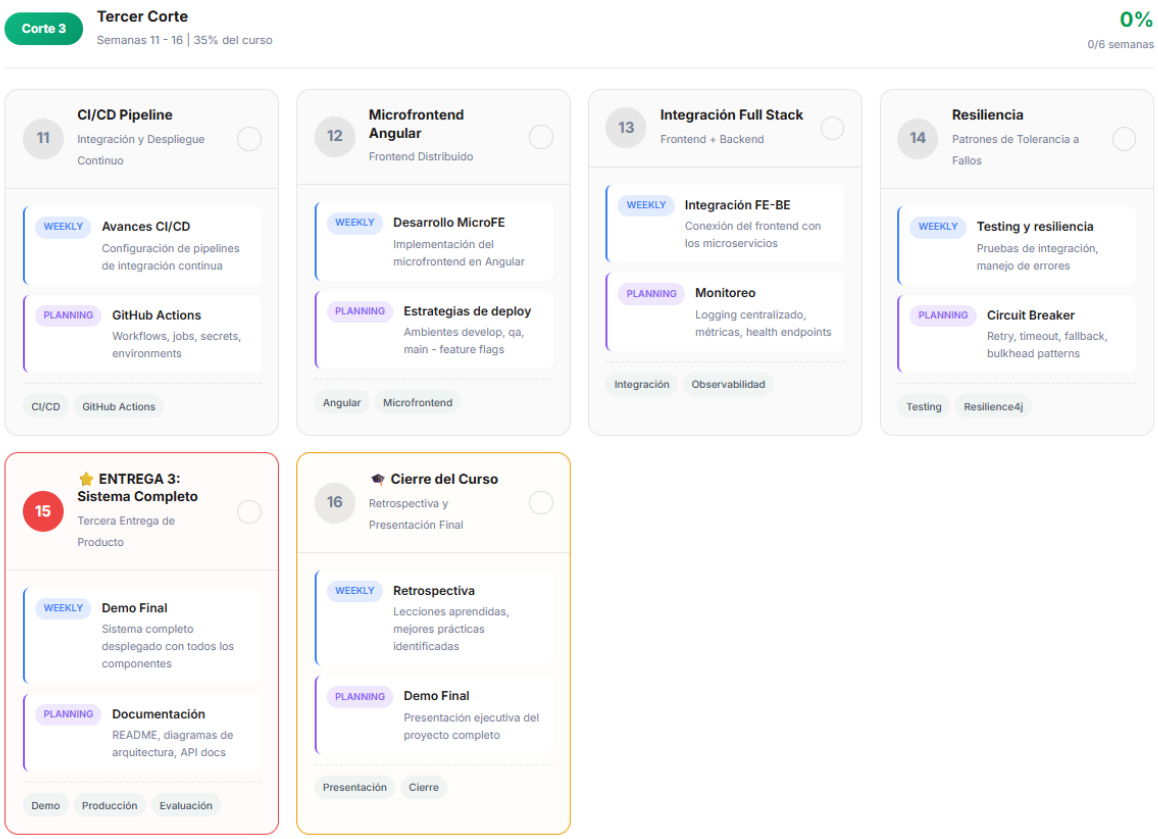
- WEEKLY — **Dockerización completa**: Todos los servicios en contenedores Docker
- PLANNING — **Docker Compose**: Orquestación local, redes, volúmenes, health checks

Docker · Docker Compose

10 ⭐ ENTREGA 2: Multi-Servicios
Segunda Entrega de Producto

- WEEKLY — **Demo Multi-servicios**: Sistema con múltiples microservicios comunicándose
- PLANNING — **Retroalimentación**: Evaluación técnica, deuda técnica, plan tercer corte

Demo · Evaluación



Unidad 1: Fundamentos y Arquitecturas en Sistemas Distribuidos

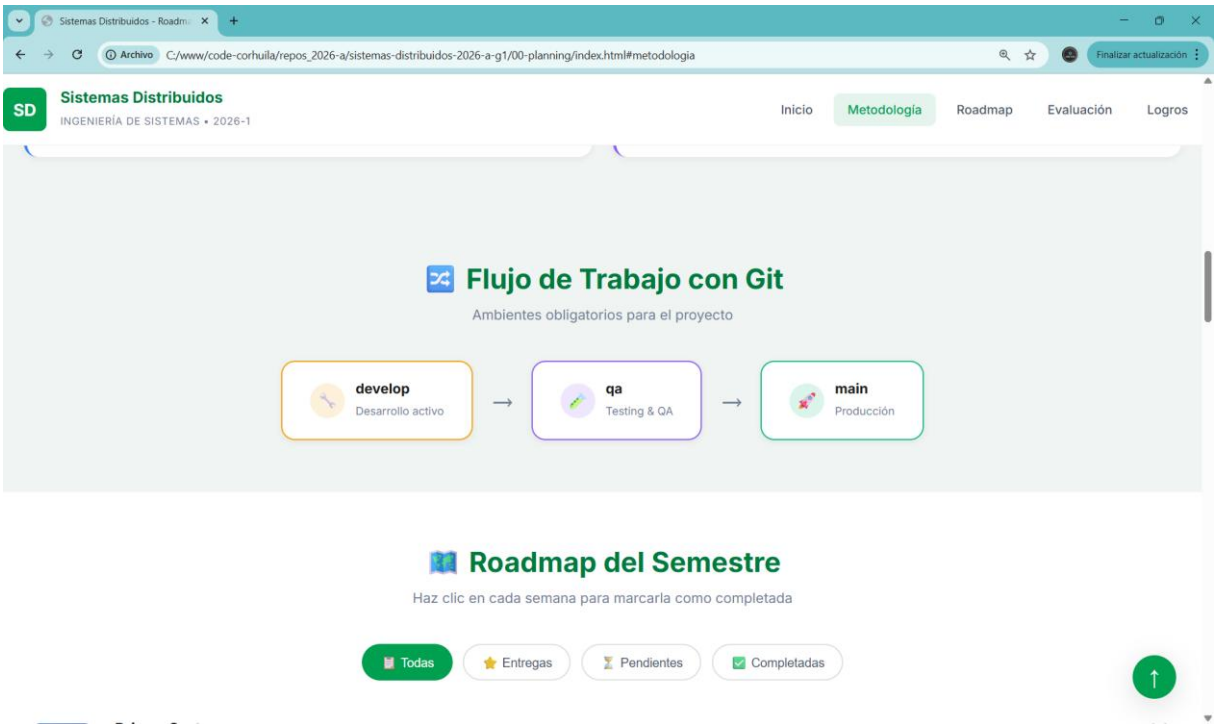
Objetivo: Establecer las bases conceptuales y arquitectónicas de los sistemas distribuidos.

Temas desarrollados:

- **Introducción a los Sistemas Distribuidos:** características, ventajas y desafíos.
- **Fundamentos de Arquitecturas Distribuidas:** modelos Cliente-Servidor, P2P y SOA.
- **Patrones de Arquitectura:** monolito, microservicios y arquitectura orientada a eventos.
- **Microservicios:** conceptos, principios de diseño e implementación base.
- Definición del caso de proyecto y primer MVP (Minimum Viable Product).

Actividades prácticas:

- Acuerdos pedagógicos y configuración del entorno de desarrollo.
- Ejercicios comparativos de arquitecturas distribuidas.
- Diseño inicial de la arquitectura del proyecto del semestre.
- Taller de definición e implementación del microservicio base.
- **Entrega MVP 1:** microservicio básico funcional con Docker.



Unidad 2: Contenerización, Redes y Persistencia

Objetivo: Implementar soluciones de contenerización, comunicación entre servicios y gestión de datos distribuidos.

Temas desarrollados:

- **Contenerización con Docker:** imágenes, contenedores y volúmenes.
- Docker Compose y orquestación básica de múltiples servicios.
- **Redes y comunicación inter-servicios:** REST APIs, gRPC y mensajería asíncrona.
- **Metodologías ágiles en proyectos distribuidos:** Scrum, Kanban y DevOps.
- **Gestión de configuración y entornos:** variables de entorno, archivos .env y secretos.

Actividades prácticas:

- Instalación y configuración del entorno Docker por equipos.
- Levantamiento de múltiples servicios con Docker Compose.
- Integración de comunicación entre microservicios del proyecto.
- Sprint planning semanal y ejecución de backlog con revisión de avance.
- **Entrega MVP 2:** sistema con comunicación inter-servicios y persistencia de datos.

Unidad 3: Operaciones, Seguridad y Proyecto Final

Objetivo: Implementar operaciones avanzadas de producción: monitoreo, seguridad, despliegue en la nube y entrega continua.

Temas desarrollados:

- **Persistencia de datos en sistemas distribuidos:** base de datos por microservicio.
- **Monitoreo y logging:** Prometheus, Grafana y ELK Stack.
- **Despliegue en la nube:** plataformas cloud y Kubernetes con namespaces por entorno.

- **Seguridad en sistemas distribuidos:** autenticación, autorización y JWT.
- **Integración y Despliegue Continuo (CI/CD):** GitHub Actions y GitLab CI.
- Presentaciones finales, pruebas de carga y revisión de resiliencia del sistema.

Actividades prácticas:

- Workshop de bases de datos distribuidas y persistencia por microservicio.
- Implementación de monitoreo y dashboards del proyecto en Grafana.
- Despliegue del proyecto en entorno de producción cloud.
- Configuración de pipeline CI/CD en el repositorio del proyecto.
- **Entrega MVP 3:** sistema completo con monitoreo, seguridad y documentación.

5. Tecnologías y Herramientas

El curso trabaja con un stack tecnológico moderno que refleja las demandas reales de la industria del software distribuido:

Área	Tecnologías y Herramientas
Lenguajes y Runtimes	Node.js + TypeScript, Spring Boot (Java), Python
Arquitecturas y Patrones	Microservicios, API Gateway, Repository, DTO, Abstract Factory, Event-Driven
Contenerización y Orquestación	Docker, Docker Compose, Kubernetes (namespaces dev/qa/main)
Bases de Datos	PostgreSQL (instancias por entorno: puertos 5432 / 5433 / 5434)
Monitoreo y Observabilidad	Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana)
Seguridad	JWT, API Gateway perimetral, variables de entorno y secretos
CI/CD y Control de Versiones	GitHub Actions, GitLab CI, estrategia de ramas dev / qa / main

6. Proyecto Integrador — Shopping Cart Distribuido

A lo largo del semestre, los estudiantes construyen un sistema de gestión de órdenes de compra distribuido denominado Shopping Cart Distribuido, que integra todos los conceptos del curso en un sistema real con tres entornos de despliegue simultáneos.

6.1 Módulos del Sistema

- **Módulo de Seguridad:** Registro de usuarios, autenticación con JWT y control de autorización por roles.
- **Módulo de Ventas:** Creación de pedidos, facturación y gestión de detalle de transacciones.
- **Módulo de Inventario:** Gestión de productos y control de stock en tiempo real.

6.2 Evolución por MVPs

Entrega	Alcance	Evidencias Clave
MVP 1	Microservicio base funcional con contenerización Docker	Imagen Docker, endpoints REST, repositorio GitHub
MVP 2	Sistema con comunicación inter-servicios, persistencia y orquestación	Docker Compose, BD por servicio, APIs integradas
MVP 3	Sistema completo con monitoreo, seguridad, CI/CD y despliegue en nube	Pipeline CI/CD, dashboards Grafana, JWT, despliegue cloud

6.3 Documentación Arquitectónica — ADRs

Cada integrante del equipo debe producir un mínimo de 5 Architecture Decision Records (ADRs) documentando decisiones de diseño, aplicación de patrones de software y mejoras de calidad del sistema. Los ADRs comprenden áreas como:

- **Patrones de diseño aplicados:** creacionales, estructurales y de comportamiento.
- Estrategias de comunicación entre servicios.
- Decisiones de persistencia y consistencia de datos.
- Mecanismos de resiliencia y tolerancia a fallos.
- Configuración de seguridad y gestión de identidad.

7. Perfil de Egreso y Competencias Desarrolladas

Al finalizar la asignatura, el estudiante estará en capacidad de:

4. Diseñar e implementar arquitecturas de microservicios para sistemas empresariales reales.
5. Contenerizar y orquestar servicios usando Docker y Kubernetes en múltiples entornos.
6. Establecer pipelines de CI/CD para automatizar la integración y el despliegue de software.
7. Implementar estrategias de monitoreo, logging y observabilidad en sistemas distribuidos.
8. Aplicar mecanismos de seguridad (autenticación y autorización) en APIs distribuidas.
9. Tomar decisiones arquitectónicas documentadas y justificadas mediante ADRs.
10. Trabajar colaborativamente con flujos de trabajo basados en Git y metodologías ágiles.

8. Bibliografía Principal

Textos de referencia obligatoria y complementaria:

- **Tanenbaum, A. S. & Van Steen, M. (2017). Distributed Systems:** Principles and Paradigms (3.a ed.). Pearson.
- **Coulouris, G. et al. (2011). Distributed Systems:** Concepts and Design (5.a ed.). Addison-Wesley.
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- Newman, S. (2021). Building Microservices (2.a ed.). O'Reilly Media.
- Richardson, C. (2018). Microservices Patterns. Manning Publications.
- **Burns, B., Beda, J. & Hightower, K. (2022). Kubernetes:** Up and Running. O'Reilly Media.
- **Mardan, A. (2014). Practical Node.js:** Building Real-World Scalable Web Apps. Apress.
- Lopez Fuentes, F. (2015). Sistemas Distribuidos. Alfaomega.